

# CS2030S

AY20/21 SEM 2

Programming Methodology II

## 01 OOP Fundamentals

### Encapsulation

Bundle data and methods within a single unit (class). Hide internal state.

```
class Circle {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    public double getRadius() { return radius; }
    public double getArea() {
        return Math.PI * radius * radius;
    }
}
```

### Immutability

Object state cannot be modified after creation. Use final.

```
class ImmutablePoint {
    private final double x;
    private final double y;

    public ImmutablePoint(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public ImmutablePoint move(double dx,
        double dy) {
        return new ImmutablePoint(x + dx, y + dy);
    }
}
```

### Tell, Don't Ask

Tell objects what to do; don't ask for their state.

```
// Bad
if (circle.getRadius() > 5) { /* ... */ }
```

```
// Good
circle.isLargerThan(5.0)
```

## 02 Inheritance & Polymorphism

### Inheritance

```
class Shape {
    private String color;
    public Shape(String color) {
        this.color = color;
    }
    double getArea() { return 0; }
}
```

```
class Circle extends Shape {
```

```
    private double radius;

    public Circle(double radius, String color)
        → {
        super(color);
        this.radius = radius;
    }

    @Override
    double getArea() {
        return Math.PI * radius * radius;
    }
}
```

### Polymorphism

Same interface, different implementations.

```
Shape s1 = new Circle(5.0, "red");
Shape s2 = new Rectangle(3.0, 4.0, "blue");
```

```
// Polymorphic method call
double area1 = s1.getArea(); // Circle's getArea
double area2 = s2.getArea(); // Rectangle's
    → getArea
```

### Abstract Classes

Cannot be instantiated; may have abstract methods.

```
abstract class Shape {
    abstract double getArea();
    abstract double getPerimeter();
}
```

```
// Concrete method
public void printInfo() {
    System.out.println("Area: " +
        → getArea());
}
}
```

### Interfaces

Contract specifying what a class must do.

```
interface Drawable {
    void draw();
}
```

```
interface Comparable<T> {
    int compareTo(T other);
}
```

```
class Circle implements Drawable,
    → Comparable<Circle> {
    public void draw() { /* ... */ }
    public int compareTo(Circle other) {
        return Double.compare(this.radius,
            → other.radius);
    }
}
```

### Liskov Substitution Principle (LSP)

Subtype must be substitutable for supertype without altering correctness.

```
// If S is subtype of T, then objects of type T
// can be replaced with objects of type S
```

```
void processShape(Shape s) {
    double area = s.getArea(); // works for all
}

```

```
processShape(new Circle(5.0));
processShape(new Rectangle(3.0, 4.0));
```

## 03 Generics

### Generic Class

```
class Box<T> {
    private T item;

    public Box(T item) {
        this.item = item;
    }

    public T getItem() {
        return item;
    }

    public <U> Box<U> map(Transformer<T, U> f)
        → {
        return new Box<>(f.transform(item));
    }
}
```

```
Box<String> strBox = new Box<>("hello");
Box<Integer> intBox = new Box<>(42);
```

### Generic Methods

```
class Utils {
    public static <T> T getFirst(List<T> list)
        → {
        return list.get(0);
    }

    public static <T extends Comparable<T>>
        T max(T a, T b) {
        return a.compareTo(b) > 0 ? a : b;
    }
}
```

### Bounded Type Parameters

```
// Upper bound: T must be Number or subclass
class NumBox<T extends Number> {
    private T value;

    public double getDouble() {
        return value.doubleValue();
    }
}
```

```
// Multiple bounds
<T extends Shape & Drawable>
```

### Wildcards

```
// Unbounded wildcard
void printList(List<?> list) {
    for (Object o : list) {
        System.out.println(o);
    }
}
```

```
}

// Upper bounded (covariant)
void processShapes(List<? extends Shape>
    → shapes) {
    for (Shape s : shapes) {
        s.getArea(); // can read
    }
    // shapes.add(new Circle()); // CANNOT add
}
```

```
// Lower bounded (contravariant)
void addCircles(List<? super Circle> list) {
    list.add(new Circle(5.0)); // can add
    // Circle c = list.get(0); // CANNOT get as
    → Circle
}
```

### PECS: Producer Extends, Consumer Super

? extends T for **reading** (producer)  
? super T for **writing** (consumer)

## 04 Functional Programming

### Lambda Expressions

```
// Syntax: (parameters) -> expression
Function<Integer, Integer> square = x -> x * x;
Predicate<Integer> isEven = x -> x % 2 == 0;
BiFunction<Integer, Integer, Integer> add =
    (a, b) -> a + b;
```

```
// Multi-line lambda
Function<String, String> process = s -> {
    String trimmed = s.trim();
    return trimmed.toUpperCase();
};
```

### Functional Interfaces

Interface with exactly one abstract method (SAM).

```
@FunctionalInterface
interface Transformer<T, R> {
    R transform(T t);
}
```

```
@FunctionalInterface
interface Predicate<T> {
    boolean test(T t);
}
```

```
// Common ones in java.util.function:
// Function<T,R>, Predicate<T>, Consumer<T>
// Supplier<T>, BiFunction<T,U,R>
```

### Method References

```
// Static method
Function<String, Integer> parse =
    → Integer::parseInt;
```

```
// Instance method of particular object
String str = "hello";
Supplier<String> toUpper = str::toUpperCase;
```

```
// Instance method of arbitrary object
Function<String, Integer> len = String::length;

// Constructor
Supplier<List<String>> listMaker =
↳ ArrayList::new;
Function<Integer, Circle> circleMaker =
↳ Circle::new;
```

Currying

Transform multi-argument function into sequence of single-argument functions.

```
// Normal function
BiFunction<Integer, Integer, Integer> add =
(x, y) -> x + y;

// Curried version
Function<Integer, Function<Integer, Integer>>
curriedAdd = x -> y -> x + y;

// Usage
Function<Integer, Integer> add5 =
↳ curriedAdd.apply(5);
int result = add5.apply(3); // 8
```

05 Streams

Creating Streams

```
// From collection
List<Integer> list = List.of(1, 2, 3, 4);
Stream<Integer> stream1 = list.stream();

// From values
Stream<String> stream2 = Stream.of("a", "b",
↳ "c");

// From array
int[] arr = {1, 2, 3};
IntStream stream3 = Arrays.stream(arr);

// Infinite streams
Stream<Integer> infinite = Stream.iterate(0, x
↳ -> x + 1);
Stream<Double> random =
↳ Stream.generate(Math::random);

// Range
IntStream range = IntStream.range(1, 10); // 1
↳ to 9
IntStream rangeClosed =
↳ IntStream.rangeClosed(1, 10); // 1 to 10
```

Intermediate Operations

Lazy; return a stream.

```
stream.filter(x -> x > 5) // keep
↳ elements
.map(x -> x * 2) //
↳ transform
.flatMap(x -> Stream.of(x, x)) // flatten
.distinct() // remove
↳ duplicates
```

```
.sorted() // natural
↳ order
.sorted(Comparator.reverseOrder())
.limit(10) // first
↳ 10
.skip(5) // skip
↳ first 5
.peek(System.out::println); // side
↳ effect
```

Terminal Operations

Eager; trigger computation and return result.

```
// Collectors
List<T> list =
↳ stream.collect(Collectors.toList());
Set<T> set =
↳ stream.collect(Collectors.toSet());
String joined =
↳ stream.collect(Collectors.joining(", "));

// Reduction
Optional<Integer> sum = stream.reduce((a, b)
↳ -> a + b);
int sum = stream.reduce(0, (a, b) -> a + b);

// Matching
boolean anyMatch = stream.anyMatch(x -> x > 5);
boolean allMatch = stream.allMatch(x -> x > 0);
boolean noneMatch = stream.noneMatch(x -> x <
↳ 0);

// Finding
Optional<T> first = stream.findFirst();
Optional<T> any = stream.findAny();

// Counting
long count = stream.count();

// ForEach
stream.forEach(System.out::println);

// Numeric operations (IntStream, etc.)
int sum = intStream.sum();
OptionalDouble avg = intStream.average();
OptionalInt max = intStream.max();
```

```
// Finding
Optional<T> first = stream.findFirst();
Optional<T> any = stream.findAny();

// Counting
long count = stream.count();

// ForEach
stream.forEach(System.out::println);

// Numeric operations (IntStream, etc.)
int sum = intStream.sum();
OptionalDouble avg = intStream.average();
OptionalInt max = intStream.max();
```

Stream Pipeline Example

```
List<Student> students = /* ... */;

// Get names of top 5 students with score > 80
List<String> topStudents = students.stream()
.filter(s -> s.getScore() > 80)
.sorted(Comparator.comparing(Student::getS
↳ core)
.reversed())
.limit(5)
.map(Student::getName)
.collect(Collectors.toList());
```

Collectors

```
// Group by
Map<String, List<Student>> byMajor =
```

```
students.stream()
.collect(Collectors.groupingBy(
Student::getMajor));

// Partition by (boolean)
Map<Boolean, List<Student>> passing =
students.stream()
.collect(Collectors.partitioningBy(
s -> s.getScore() >= 50));

// Summarizing
IntSummaryStatistics stats =
students.stream()
.collect(Collectors.summarizingInt(
Student::getScore));
System.out.println(stats.getAverage());
```

06 Exception Handling

Try-Catch-Finally

```
try {
int result = riskyOperation();
} catch (SpecificException e) {
// Handle specific
} catch (Exception e) {
// Handle general
} finally {
// Always executed (cleanup)
}
```

Checked: Must be caught/declared (Exception)  
Unchecked: Runtime exceptions (RuntimeException)

Try-with-Resources

```
try (FileReader fr = new FileReader("f.txt")) {
String line = br.readLine();
} // Resources auto-closed
```

07 CompletableFuture

Basics & Transformations

```
CompletableFuture<String> cf =
CompletableFuture.supplyAsync(() ->
↳ "Hello");

cf.thenApply(x -> x.length()) // sync
.thenApplyAsync(x -> x + 1) // async
.thenCompose(x -> // flatten
CompletableFuture.supplyAsync(() -> x *
↳ 3))
.exceptionally(ex -> 0); // error
```

Combining

```
cf1.thenCombine(cf2, (a, b) -> a + b); //
↳ combine
cf1.applyToEither(cf2, x -> x * 2); // first
CompletableFuture.allOf(cf1, cf2); // all
CompletableFuture.anyOf(cf1, cf2); // any
```

08 Design Principles

SOLID: Single Responsibility, Open/Closed, Liskov  
Substitution, Interface Segregation, Dependency Inversion

Composition over Inheritance

```
// Composition (flexible)
class ColoredShape {
private Shape shape;
private String color;

public double getArea() {
return shape.getArea();
}
}
```

Law of Demeter

Only call methods on: self, parameters, created objects,  
instance variables.

```
// Bad
customer.getWallet().getMoney().getAmount();
// Good
customer.pay(amount);
```

09 Optional

```
// Create
Optional<String> opt = Optional.of("value");
Optional<String> opt =
↳ Optional.ofNullable(val);
Optional<String> opt = Optional.empty();

// Use
opt.isPresent(); opt.get();
opt.ifPresent(v -> System.out.println(v));

// Get with default
opt.orElse("default");
opt.orElseGet(() -> compute());
opt.orElseThrow(() -> new Exception());

// Transform
opt.map(String::length);
opt.flatMap(s -> Optional.of(s.length()));
opt.filter(s -> s.length() > 5);
```

10 Key Concepts

Object Methods

```
@Override
public boolean equals(Object obj) {
if (this == obj) return true;
if (obj == null || getClass() !=
↳ obj.getClass())
return false;
Circle o = (Circle) obj;
return Double.compare(radius, o.radius) ==
↳ 0;
}

@Override
public int hashCode() {
return Objects.hash(radius);
}
```

Type Erasure: Generics removed at runtime. Cannot: new  
T(), new T[], T.class  
Autoboxing: int ↔ Integer (heap allocation cost)